

Brenden's Arcane Archive — Project Spec Sheet

1. Project Overview

Project name: Brenden's Arcane Archive

Motto: *Not all magic is cast. Some is remembered.*

Brenden's Arcane Archive is a personal photography website presented as a magical library. The site is not fantasy for fantasy's sake. It is an archive: a structured, searchable, story-rich home for photographs, collections, metadata, and personal narratives.

The magical layer is an interface language, not the subject itself. The site uses spellcraft, runes, circles, glyphs, and subtle VFX to make discovery feel meaningful, but the actual content remains grounded in real photography: people, nature, moments, scenes, and memories.

The project has two separate but interacting systems:

1. **Content architecture** — the archive, collections, photos, metadata, tags, story text, search.
2. **Visual architecture** — the magical UI language, motion, glyphs, spell effects, transitions, and atmosphere.

These systems must remain modular so the archive can scale without the visuals becoming brittle.

2. North Star

Brenden's Arcane Archive is a living library of preserved memories, where magic is the language used to discover, organize, and remember real photographs.

Every decision should support that idea.

If a feature does not help preserve a memory, reveal a story, or help a visitor navigate the archive, it does not belong.

3. Core Goals

Primary goals

- Present photography in a unique, memorable, and personal way.
- Make browsing feel like exploring a magical archive.
- Keep navigation clean, scalable, and fast.
- Support large and growing collections.
- Support future search, filtering, and storytelling.

- Keep the site lightweight enough for GitHub Pages.
- Keep content and visuals decoupled so both can evolve independently.

Secondary goals

- Provide a desktop app for maintaining the archive.
 - Automate image conversion and metadata generation.
 - Make adding, editing, and organizing content efficient.
 - Allow future growth into more advanced filtering, search, and thematic sorting.
-

4. Creative Direction

Inspiration

The site should be inspired by:

- magical libraries
- spellbooks and grimoires
- rune circles
- arcane geometry
- illuminated manuscripts
- bioluminescent light
- subtle enchantment
- ritualized discovery
- quiet wonder

Emotional tone

The site should feel:

- magical
- whimsical
- grounded
- refined
- calm
- handcrafted
- slightly mysterious
- premium
- personal
- intentional

What it should not feel like

- a childish fantasy skin
- a gimmick site
- a game menu
- a cluttered art project

- a novelty portfolio with hard-to-use navigation
-

5. Metaphor System

Mapping of concepts

- **Site** → Archive / library
- **Collections** → Volumes / spellbooks / grimoires
- **Photos** → Pages / observations / preserved moments
- **Tags** → Runes / affinities / attributes
- **Search** → Casting a spell
- **Filter** → Focus / attunement
- **Sort** → Alignment
- **Description / story** → Memory / narration / archive note
- **Private note** → Curator note

Why this mapping works

The library metaphor gives structure. The spellcraft metaphor gives personality. The photography itself remains authentic.

This allows the site to support many categories and styles of work without forcing all photos into a single visual or narrative box.

6. Design Rules

Rule 1: The archive is real; the magic is the interface

Do not make the photography itself look fantasy-themed. The magic belongs in:

- navigation
- filtering
- search
- hover states
- transitions
- collection framing
- reveal moments

Rule 2: Magic should be restrained

Use glow, glyphs, circles, and motion with intent. Do not saturate the UI with effects.

Rule 3: Content and visuals stay separate

The content engine knows nothing about glow or spell circles.

The visual engine knows nothing about photography categories beyond the data it receives.

Rule 4: Browsing must stay fast and shallow

Visitors should not need to click through several layers just to reach images.

Rule 5: The site must scale cleanly

The architecture must support more collections, more tags, and more photos without redesigning the system.

Rule 6: The desktop app is a tool

The desktop app should be practical, fast, and lightweight. It should not try to be the magical experience.

Rule 7: The website must stay lightweight

Because it will live on GitHub Pages and serve many images, performance matters.

7. Information Architecture

Top-level structure

- Home
- Library
- Collections
- Cast / Search
- About / Archive Notes

Home

The homepage introduces the archive, the motto, and a strong visual entry point into the library.

Library

The main browsing area. This is where collections appear as books or volumes on shelves.

Collections

Each collection is a self-contained volume with its own story, tags, photos, and metadata.

Cast / Search

The spellcasting interface for finding, filtering, and exploring the archive.

About / Archive Notes

A small explanatory page that introduces the archive philosophy and the meaning behind the system.

8. Content Model

The content model is the source of truth for the archive.

Collection

A collection should include:

- title
- slug
- volume number
- cover image
- description
- story
- tags
- mood
- date range
- location or region
- featured flag
- visibility flag
- photo order

Photo

Each photo should include:

- filename
- title
- caption
- story / narration
- curator note
- tags
- mood
- date taken
- location
- camera metadata
- lens metadata
- exposure metadata

- orientation
- dimensions
- alt text
- generated image variants

Metadata layers

1. Technical metadata

Extracted automatically from the source image:

- camera
- lens
- ISO
- shutter speed
- aperture
- focal length
- timestamp
- GPS if available
- dimensions

2. Archive metadata

Used by the site and search:

- collection
- volume
- tags
- mood
- order
- cover
- featured status
- visibility

3. Story metadata

Used to tell the memory behind the image:

- title
- caption
- story text
- narrative context

4. Private curator notes

For personal workflow only:

- favorite

- revisit later
- print candidate
- retouch later
- competition candidate
- archive note

This separation keeps the archive clean and future-proof.

9. Storytelling Model

The archive should support narration, not just captions.

Photo-level story

Each image can have:

- a short caption
- a longer narrative
- personal context
- an optional memory note

Collection-level story

Each collection can have:

- an introduction
- a broader narrative
- chapter-like section markers
- an emotional throughline

Why this matters

Photography is not only about how an image looks. It is about why it matters. The storytelling layer preserves that meaning.

10. Visual System

Visual identity

The UI should feel like magical scholarship:

- dark
- luminous
- precise
- organic

- elegant
- atmospheric

Materials and motifs

Use visual cues from:

- light
- glass
- ink
- bronze
- stone
- mist
- parchment
- energy lines
- circles
- sigils
- thin glyphs

Color direction

Use a restrained palette:

- deep charcoal
- moss-shadow green
- ink black
- muted brass
- cyan glow
- pale parchment text
- soft amber highlights

Typography

The typography should balance:

- elegance
- readability
- modernity

Use a refined heading style and a clean body style. The magic should come from the overall composition, not from overly decorative text.

Shape language

Prefer:

- circles
- rings

- rounded rectangles
 - framed panels
 - soft borders
 - layered cards
 - subtle dividers
 - sigil-like accents
-

11. Spellcasting Interaction Language

The archive should feel interactive in a magical way.

User-facing spell language

- Search → Cast
- Filter → Focus
- Sort → Align
- Open collection → Unseal
- Reset filters → Dispel
- Reveal hidden volumes → Illuminate
- Discover similar photos → Attune
- Random browse → Summon

Search behavior

The search field should feel like a spell input. Typing should create a subtle feeling of completing an incantation:

- glyph ring reaction
- soft pulse
- collection highlights
- shelves reacting to results

Filter behavior

Filters should feel like targeted magical focus, not generic UI toggles.

Important restraint

Use spell behavior where it transforms discovery. Do not force magical motion into every control.

12. Collection Presentation

Collection cards

Each collection card should feel like a volume on a shelf:

- title
- volume number
- cover image
- short teaser
- tags
- subtle sigil or rune
- mood indicator if needed

Collection pages

Each collection page should feel like reading through a volume:

- title
- volume label
- date range
- intro story
- photo sections
- optional chapter markers
- scroll-based reading experience
- simple table of contents if needed

Chapters

Chapters should be optional and lightweight. They should help organize stories, not force extra clicking.

13. Navigation Philosophy

The archive should be easy to traverse

The magical theme must not create friction.

Visitors should be able to:

- open the library
- skim collections
- open one quickly
- read the story
- browse the photos
- return to the library

Keep it shallow

Avoid extra nesting unless it serves the archive.

Keep it searchable

Tags and search should support future growth from the start.

14. Website Tech Stack

Recommended stack

- **React**
- **TypeScript**
- **Vite**
- **Tailwind CSS**
- **CSS variables**
- **Motion** for animation
- **SVG** for runes, circles, dividers, and glyphs
- **GitHub Pages** for hosting
- **GitHub Actions** only for deployment, not content generation

Why React + Vite

- fast to develop
- easy to scale
- component-based
- efficient for reusable UI
- friendly to GitHub Pages
- easy to combine with motion and SVG effects

Why Tailwind + CSS variables

- fast iteration
- consistent spacing and structure
- easy theme control
- simple to maintain as the archive grows

Why Motion + SVG

- Motion handles transitions and interaction well
- SVG is perfect for spell circles, glyphs, sigils, and line art
- the system stays lightweight and visually precise

Why not plain HTML/CSS/JS

Plain HTML/CSS/JS is possible, but the project will grow. React gives structure and maintainability for collection views, filtering, stories, and future feature expansion.

15. Hosting Strategy

Primary hosting choice

GitHub Pages

This is the right choice because:

- the site is static
- it fits your developer profile
- it is easy to access and share
- it lives naturally in GitHub
- it requires no backend
- it supports a repo-based archive workflow

Deployment philosophy

The website should never build archive content on its own. It should only render what already exists in the repository.

GitHub Actions role

GitHub Actions should be used only to:

- deploy the site
- validate that content files are valid if needed
- build the React app
- publish static output to GitHub Pages

GitHub Actions should not be the authoring system.

16. Desktop App Strategy

The desktop app is your working tool for creating and maintaining the archive.

Role of the desktop app

The desktop app should:

- read the repository
- display existing collections and photos
- edit metadata and story text
- import high-quality source images
- generate WebP variants
- create thumbnails and responsive sizes
- validate content before publish
- commit and push changes back to GitHub
- keep a local cache of repository data

Desktop app philosophy

It should be light, fast, and utilitarian. No fancy magic UI. It is the workbench, not the showpiece.

Recommended desktop stack

- **.NET 8**
- **Avalonia UI**

Why Avalonia

- lightweight compared with Electron
- cross-platform
- good for a practical archive tool
- modern enough to feel current
- fits your desire for a non-bloated laptop-friendly app

Why not Electron

Electron is heavy and unnecessary for your needs. This is a utility app, not a large packaged browser shell.

17. Desktop App Features

Core features

- browse collections
- create a new collection
- edit collection metadata
- edit photo metadata
- edit story text
- edit curator notes
- drag and drop image import
- auto-read EXIF data

- auto-generate WebP variants
- generate thumbnails and previews
- assign tags and mood
- set covers
- order photos inside a collection
- validate archive content
- commit and push to GitHub

Nice-to-have features

- batch editing
- tag suggestions
- search through archive content
- favorite / private notes system
- quick collection switching
- diff view for edited metadata
- preview website output locally

UI style

- clean lists
- fast forms
- no unnecessary motion
- minimal visual decoration
- keyboard-friendly workflows
- clear actions and validation states

The app should feel like an archivist's desk.

18. Content Pipeline

Source of truth

The repository is the archive.

Workflow

1. Import source images into desktop app.
2. Read technical metadata.
3. Create or edit collection.
4. Add title, story, tags, mood, notes.
5. Choose cover image.
6. Convert images to WebP and generate sizes.
7. Write collection/photo data files.
8. Validate everything.
9. Commit to local Git repository.
10. Push to GitHub.

11. GitHub Pages deploys the static site.

Why this workflow works

It keeps the system simple, reproducible, and version-controlled. The archive is always stored in the repo, and the site is just a renderer.

19. Image Pipeline

Because the site is image-heavy, the image pipeline needs to be efficient.

Requirements

- convert JPEG to WebP
- generate responsive sizes
- generate thumbnails
- preserve quality
- keep file size low
- keep consistent naming

Suggested sizes

- thumbnail
- small
- medium
- large
- full or original if needed

Why WebP

WebP is ideal for a static gallery because it reduces file size without sacrificing too much quality.

Rule

Never manually prepare every image by hand inside the website code. The desktop app should generate the assets.

20. Data Storage Model

Recommended storage approach

Use file-based content in the repository.

Suggested structure

- `content/collections/<collection-slug>/collection.json`
- `content/collections/<collection-slug>/photos/*.json`
- `content/collections/<collection-slug>/images/*.webp`
- `content/collections/<collection-slug>/thumbs/*.webp`

Optional manifest

A root archive manifest can store:

- archive version
- last generated time
- number of collections
- number of photos
- generated checksum
- search index version

This helps the app and site understand the archive state.

21. Caching and Sync Strategy

Desktop app sync model

The desktop app should:

- clone the repo locally
- pull changes from GitHub
- cache the repository data locally
- only regenerate changed assets
- only rewrite touched content

Why use Git over raw API calls

Git is better for:

- version history
- diffs
- conflict handling
- branch support
- efficient synchronization
- reliable local caching

Raw API use

The GitHub raw API can be used for reading public files if needed, but it should not be the primary architecture.

Recommended approach

Use Git as the primary sync mechanism. The repository itself is the database.

22. Technical Architecture

The project should be split into clear systems.

1. Archive System

Responsible for:

- collections
- photos
- tags
- metadata
- stories
- manifest
- search index

2. Rendering System

Responsible for:

- routing
- page layout
- collection rendering
- photo display
- responsive behavior

3. Arcane System

Responsible for:

- spell circles
- glyphs
- glow
- particles
- transitions
- magical framing
- atmosphere

4. Interaction System

Responsible for:

- hover
- focus
- filter behavior
- keyboard shortcuts
- accessibility
- transitions between archive states

Why this separation matters

Each system can evolve independently, which makes maintenance much easier.

23. Component Philosophy

The visual system should be built from reusable components with consistent naming.

Examples:

- SpellCircle
- RuneDivider
- GlyphButton
- ArchiveCard
- VolumeCard
- AttunementToggle
- FocusPanel
- IncantationSearch
- SigilIcon
- MemoryFrame

These are not just decorative names. They help organize the codebase around a coherent visual vocabulary.

24. Accessibility and Usability

Even with a magical aesthetic, usability must remain strong.

Accessibility requirements

- readable contrast
- keyboard navigation
- focus states
- semantic structure

- alt text for every image
- no effect should block access to content
- motion should be restrained and optional where possible

Usability requirements

- fast browsing
 - clear collection hierarchy
 - easy return to the library
 - clear search/filter feedback
 - no hidden navigation logic
-

25. Performance Budgets

Because the site is image-heavy and hosted statically, performance matters.

Budgets

- keep JS lightweight
- keep animations efficient
- avoid heavy runtime image processing
- preload only what is necessary
- use responsive images
- avoid unnecessary third-party dependencies

Practical performance rules

- use WebP
 - lazy-load offscreen images
 - generate thumbnails for previews
 - avoid huge animated overlays
 - keep effects subtle and reusable
-

26. Naming and Language Guidelines

Use language that reinforces the archive identity.

Good terms

- archive
- library
- volume
- collection
- grimoire
- page

- story
- memory
- preserved
- cast
- focus
- reveal
- attune
- illuminate

Tone rule

Use magical language where it strengthens the experience, but keep the overall tone grounded and coherent.

27. Art Direction Guidelines

Do

- make the archive feel ancient but the software feel modern
- use soft magical light
- let the photos remain the brightest objects
- use rune circles as functional motifs
- make the search and filter flow feel like casting
- use subtle storytelling and section breaks
- make each collection feel like a volume with identity

Don't

- overdecorate
 - turn the archive into a game
 - use too many particle effects
 - make the app visually noisy
 - hide the photography behind theme
 - force unnecessary extra clicks
-

28. Development Roadmap

Phase 1 — Define the system

- lock the content model
- define collection/photo JSON shape
- define naming conventions
- define folder structure
- define tags and moods
- define archive terminology

Phase 2 — Build the website foundation

- set up React + Vite + TypeScript
- set up routing
- build library pages
- build collection pages
- build search/filter base
- set up GitHub Pages deployment

Phase 3 — Build the visual language

- create design tokens
- create spell motifs
- build rune dividers and circles
- create card and panel styles
- add subtle motion

Phase 4 — Build the content pipeline

- create desktop app project
- implement image import
- implement EXIF extraction
- implement metadata editing
- implement WebP conversion
- implement JSON generation

Phase 5 — Connect desktop app to Git

- sync local repo clone
- read existing archive content
- push updates safely
- add validation before publish

Phase 6 — Polish the archive

- refine search casting
- refine hover effects
- refine story presentation
- refine collection covers
- refine mobile behavior
- refine accessibility

29. Definition of Done

The project is in a good state when:

- the archive structure is clean and scalable

- collections render clearly
 - search and filtering feel magical but functional
 - the desktop app can manage content efficiently
 - images are optimized and responsive
 - GitHub Pages hosts the site without extra runtime complexity
 - the content model supports stories and metadata
 - the visual identity feels unique and cohesive
 - the site remains fast and easy to maintain
-

30. Final Guiding Principle

Whenever a feature is proposed, ask:

Does this help preserve a memory or help someone discover one?

If the answer is yes, it belongs in Brenden's Arcane Archive.

If the answer is no, it probably does not.

That is the rule that should keep the project coherent for the long term.